

Vault Guardians Audit Report

Version 1.0

raihanmd.xyz

May 16, 2026

Vault Guardians Audit Report

Raihanmd

May 16, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-01] VaultShares constructor incorrectly initializes i_uniswapLiquidityToken for WETH vaults leading to permanent DoS
 - [H-02] Missing token approvals in _uniswapDivest renders Uniswap positions unliquidatable
 - [H-03] Guardian stake is not bonded, allowing guardians to redeem stake while remaining active
 - [H-04] Public rebalanceFunds() enables on-demand sandwiching and MEV extraction
 - [H-05] Dust LP token donation bricks all withdrawals (DoS)
 - [H-06] Hardcoded stake price of 10 ether makes USDC guardian registration impossible
 - [H-07] mint() bypasses custom deposit economics (fees and investment strategy)

- [H-08] No timelock in governance allows immediate execution of malicious proposals
- Medium
 - [M-01] Immutable Aave Pool Address Prevents Adapter Migration When Aave Pool Changes
 - [M-02] Missing Aave Rewards Claiming Mechanism Causes Protocol Yield Loss
 - [M-03] Governor Quorum Configuration Mismatch Can Weaken Governance Security
 - [M-04] Guardians Can Be Created Without Paying the Documented Entry Fee
 - [M-05] Repeated Guardian Registration Orphans Previously Tracked Vault
 - [M-06] Performance Fee Is Charged Against Principal Instead of Profit
- Low
 - [L-01] Missing Token Uniqueness Validation Allows Duplicate Supported Token Configuration
 - [L-02] GUARDIAN_FEE Constant Is Defined But Never Used
 - [L-03] i_uniswapFactory Is Stored But Not Used In UniswapAdapter
 - [L-04] _aaveDivest() Named Return Value Is Never Assigned
 - [L-05] Typographical Errors In Event Names Can Break Off-Chain Indexing
 - [L-06] GuardianAdded Event Does Not Include The Created Vault Address
 - [L-07] FundsInvested() Event Does Not Emit Useful Investment Data
 - [L-08] updateGuardianStakePrice Emits The Same Value For Old And New Stake Price
 - [L-09] updateGuardianAndDaoCut Emits The Wrong Event Type
- Informational
 - [I-01] Hard dependency on Aave/Uniswap in withdrawal and deposit paths
 - [I-02] _uniswapDivest() returns incorrect value, ignoring direct liquidity return
 - [I-03] Single-Step Ownership Transfer Can Cause Accidental Ownership Loss
 - [I-04] Temporary Swap Path Is Stored In Contract Storage

Protocol Summary

Vault Guardians is a protocol that allows users to deposit ERC20 tokens into ERC4626 vaults managed by vault guardians. The goal of a vault guardian is to manage the vault to maximize value for depositors by allocating funds across Aave V3, Uniswap V2, or simply holding the assets.

Vault guardians must stake a deposit to become guardians and can move funds between the investable universe protocols (Aave, Uniswap, or hold). The protocol includes a DAO that handles pricing parameters and receives a cut of guardian performance fees.

The protocol supports WETH, LINK, and USDC as deposit assets, with Aave V3 for lending and Uniswap V2 for liquidity provision.

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 0600272180b6b6103f523b0ef65b070064158303
- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum

Scope

- In Scope:

```
1 src/
2   abstract/
3     AStaticTokenData.sol
4     AStaticUSDCData.sol
5     AStaticWethData.sol
6   dao/
7     VaultGuardianGovernor.sol
8     VaultGuardianToken.sol
9   interfaces/
10    InvestableUniverseAdapter.sol
11    IVaultData.sol
12    IVaultGuardians.sol
13    IVaultShares.sol
14  protocol/
15    VaultGuardians.sol
16    VaultGuardiansBase.sol
17    VaultShares.sol
18    investableUniverseAdapters/
19      AaveAdapter.sol
20      UniswapAdapter.sol
21  vendor/
22    DataTypes.sol
23    IUniswapV2Factory.sol
24    IUniswapV2Router01.sol
25    IPool.sol
```

- Total: 18 Solidity files in scope

Roles

- Vault Guardian: Fund managers who stake tokens to manage vaults. They can allocate funds across Aave, Uniswap, or hold positions.
- User: Depositors who allocate funds to guardian vaults to earn yield.
- DAO: Governs protocol parameters including stake price and fee cuts.

Executive Summary

We spent significant time with 1 security researcher, using Foundry as a framework for testing.

Issues found

Severity	Number of issues found
HIGH	8
MEDIUM	6
LOW	9
INFO	4
TOTAL	27

Findings

High

[H-01] VaultShares constructor incorrectly initializes `i_uniswapLiquidityToken` for WETH vaults leading to permanent DoS

Description:

In the `VaultShares` constructor, the immutable variable `i_uniswapLiquidityToken` is initialized by querying the Uniswap V2 Factory for the pair of the vault's underlying `asset` and `weth`:

```
1 i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(
    constructorData.asset), address(i_weth)));
```

When the vault's underlying asset is WETH, this call becomes `getPair(WETH, WETH)`, which returns `address(0)` because a pair cannot consist of the same token. However, the `UniswapAdapter` logic (specifically `_uniswapInvest`) dictates that if the asset is WETH, it should be paired with USDC (`i_tokenOne`).

The `divestThenInvest` modifier, used by critical functions like `deposit`, `withdraw`, `redeem`, and `rebalanceFunds`, performs a balance check on this address:

```
1 uint256 uniswapLiquidityTokensBalance = i_uniswapLiquidityToken.
    balanceOf(address(this));
```

Impact:

Any call to `balanceOf` on `address(0)` will revert. This means that for WETH vaults, any function protected by the `divestThenInvest` modifier (which is almost all state-changing user functions) will permanently revert. WETH vaults are rendered completely non-functional from deployment.

Proof of Concept:

1. A guardian attempts to create a WETH vault by calling `becomeGuardian`.
2. The `VaultShares` constructor is called with `asset = WETH`.
3. `i_uniswapLiquidityToken` is set to `getPair(WETH, WETH)`, which is `address(0)`.
4. Any user attempts to `deposit` into the WETH vault.
5. The `divestThenInvest` modifier triggers and calls `i_uniswapLiquidityToken.balanceOf(address(this))`.
6. The transaction reverts because it attempts to call a function on `address(0)`.

Recommended Mitigation:

Update the `VaultShares` constructor to correctly identify the counterparty token when the asset is WETH, matching the logic used in `UniswapAdapter`.

```
1 address counterPartyToken = address(constructorData.asset) == address(
    constructorData.weth)
2   ? address(constructorData.usdc)
3   : address(constructorData.weth);
4
5 i_uniswapLiquidityToken = IERC20(i_uniswapFactory.getPair(address(
    constructorData.asset), counterPartyToken));
```

[H-02] Missing token approvals in `_uniswapDivest` renders Uniswap positions un-liquidatable**Description:**

The `UniswapAdapter::_uniswapDivest` function is responsible for removing liquidity from Uniswap V2 and swapping the counterparty token back to the vault's underlying asset. However, the function fails to grant the Uniswap V2 Router the necessary ERC20 allowances to perform these operations.

According to the Uniswap V2 Router implementation:

1. `removeLiquidity` calls `pair.transferFrom(msg.sender, pair, liquidity)` to pull LP tokens from the caller.
2. `swapExactTokensForTokens` calls `transferFrom` on the input token (`path[0]`) to pull funds from the caller.

The current implementation in `UniswapAdapter.sol` calls these functions without any prior `approve` calls:

```
1 function _uniswapDivest(IERC20 token, uint256 liquidityAmount) internal
  returns (uint256 amountOfAssetReturned) {
2   IERC20 counterPartyToken = token == i_weth ? i_tokenOne : i_weth;
3
4   // @audit No approval for LP token before this call
5   (uint256 tokenAmount, uint256 counterPartyTokenAmount) =
    i_uniswapRouter.removeLiquidity({
6     tokenA: address(token),
7     tokenB: address(counterPartyToken),
8     liquidity: liquidityAmount,
9     // ...
10  });
11
12  s_pathArray = [address(counterPartyToken), address(token)];
13  // @audit No approval for counterPartyToken before this call
14  uint256[] memory amounts = i_uniswapRouter.swapExactTokensForTokens
    ({
15    amountIn: counterPartyTokenAmount,
16    // ...
17  });
18  // ...
19 }
```

Impact:

Any operation that triggers a divestment from Uniswap (including `withdraw`, `redeem`, `rebalanceFunds`, and `quitGuardian`) will permanently revert if the vault has a non-zero Uniswap LP balance. This results in a complete Denial of Service (DoS) for user withdrawals and effectively locks all funds currently invested in Uniswap liquidity pools.

Proof of Concept:

```
1 function test_H02_MissingUniswapApprovalsInDivest() public {
2   // 1. Setup: Guardian with 75% Hold / 25% Uniswap allocation
3   AllocationData memory mixAlloc =
4     AllocationData({holdAllocation: 750, uniswapAllocation: 250,
5     aaveAllocation: 0});
6
7   weth.mint(MINT_AMOUNT * 2, guardian);
8   vm.startPrank(guardian);
9   weth.approve(address(vaultGuardians), MINT_AMOUNT);
10  vaultGuardians.becomeGuardian(mixAlloc);
11  vm.stopPrank();
12
13  link.mint(MINT_AMOUNT * 2, guardian);
14  vm.startPrank(guardian);
15  link.approve(address(vaultGuardians), MINT_AMOUNT);
16  address linkVaultAddr = vaultGuardians.becomeTokenGuardian(mixAlloc,
    IERC20(linkAddress));
17  VaultShares linkVault = VaultShares(linkVaultAddr);
```



```
17     vm.stopPrank();
18
19     // 2. Verify vault has LP tokens from guardian stake investment
20     address lpToken = linkVault.getUniswapLiquidityToken();
21     uint256 lpBalance = IERC20(lpToken).balanceOf(address(linkVault));
22     console2.log("LP balance in vault:", lpBalance);
23
24     // 3. Verify zero allowances (the bug!)
25     uint256 lpAllowance = IERC20(lpToken).allowance(address(linkVault),
26         address(uniswapRouter));
27     uint256 wethAllowance = weth.allowance(address(linkVault), address(
28         uniswapRouter));
29
30     console2.log("LP allowance to router:", lpAllowance);
31     console2.log("WETH allowance to router:", wethAllowance);
32
33     assertEq(lpAllowance, 0, "LP tokens NOT approved to router");
34     assertEq(wethAllowance, 0, "Counterparty NOT approved to router");
35
36     console2.log("POC H-02 Success: Vault has LP tokens but ZERO
37         allowances to Uniswap Router");
38     console2.log("On mainnet, rebalanceFunds() would revert because
39         router cannot pull tokens");
40 }
```

Recommended Mitigation:

Add the necessary `approve` calls before interacting with the Uniswap Router in `_uniswapDivest`.

```
1 address pair = i_uniswapFactory.getPair(address(token), address(
2     counterPartyToken));
3 IERC20(pair).approve(address(i_uniswapRouter), liquidityAmount);
4 // ... removeLiquidity ...
5
6 counterPartyToken.approve(address(i_uniswapRouter),
7     counterPartyTokenAmount);
8 // ... swapExactTokensForTokens ...
```

[H-03] Guardian stake is not bonded, allowing guardians to redeem stake while remaining active**Description:**

When a user becomes a vault guardian, their stake is deposited into the vault as ordinary ERC4626 shares via `tokenVault.deposit(s_guardianStakePrice, msg.sender)`. The protocol

tracks guardian status solely through the `s_guardians[guardian][token]` mapping, which is set during registration and cleared only when `quitGuardian()` is called.

Crucially, there is no mechanism that bonds or locks these shares, nor is there any check in guardian-only functions to verify that the guardian still holds the required stake.

Impact:

A guardian can call `redeem()` or `withdraw()` directly on their vault to recover 100% of their stake shares at any time, without calling `quitGuardian()`. After doing so, they remain registered as an active guardian with full privileges (e.g., `updateHoldingAllocation`) but have zero economic skin in the game. This completely bypasses the protocol's security model, which relies on the stake to align guardian incentives with user safety.

Proof of Concept:

```
1 function test_H03_GuardianStakeNotBonded() public {
2     // 1. Become WETH guardian
3     weth.mint(MINT_AMOUNT, guardian);
4     vm.startPrank(guardian);
5     weth.approve(address(vaultGuardians), MINT_AMOUNT);
6     address wethVaultAddr = vaultGuardians.becomeGuardian(
7         AllocationData(1000, 0, 0));
8     VaultShares wethVault = VaultShares(wethVaultAddr);
9     vm.stopPrank();
10
11     // 2. Record guardian status and shares
12     assertEq(
13         address(vaultGuardians.getVaultFromGuardianAndToken(guardian,
14             weth)),
15         wethVaultAddr,
16         "Guardian should be registered"
17     );
18     uint256 guardianSharesBefore = wethVault.balanceOf(guardian);
19     assertGt(guardianSharesBefore, 0, "Guardian should have shares from
20         stake");
21
22     // 3. Guardian redeems ALL shares directly (WITHOUT quitGuardian)
23     vm.startPrank(guardian);
24     wethVault.redeem(guardianSharesBefore, guardian, guardian);
25     vm.stopPrank();
26
27     // 4. Verify guardian is STILL registered after withdrawing stake
28     assertEq(
29         address(vaultGuardians.getVaultFromGuardianAndToken(guardian,
30             weth)),
31         wethVaultAddr,
32         "Guardian should STILL be registered after redeeming stake"
33     );
34 }
```

```
31 // 5. Verify guardian-only functions still work
32 AllocationData memory newAlloc = AllocationData(1000, 0, 0);
33 vm.prank(guardian);
34 vaultGuardians.updateHoldingAllocation(weth, newAlloc);
35
36 console2.log("POC H-03 Success: Guardian redeemed stake but remains
37             registered");
37 }
```

Recommended Mitigation:

Implement a bonding mechanism that prevents guardians from withdrawing their stake while registered. Options include:

- Locking the stake shares in a separate contract that only releases them upon `quitGuardian()`.
- Adding a modifier to guardian-only functions that checks `vault.balanceOf(guardian) >= minimumStakeShares`.
- Overriding `transfer/transferFrom` in the vault to block share transfers from registered guardians.

[H-04] Public `rebalanceFunds()` enables on-demand sandwiching and MEV extraction

Description:

The `rebalanceFunds()` function in `VaultShares.sol` is marked as **public**, allowing any external address to trigger a full divestment and reinvestment of the vault's Uniswap position at any time.

Combined with the known slippage vulnerability (`amountOutMin = 0` in `_uniswapInvest` and `_uniswapDivest`), this creates a highly dangerous attack vector. Unlike `deposit` or `withdraw` which require a user to initiate the transaction, `rebalanceFunds()` allows an attacker to **force** the vault into a vulnerable swap at a time of their choosing.

Impact:

1. **Permissionless Value Extraction:** An MEV bot can monitor the vault's liquidity position, manipulate the Uniswap pool price via flashloan, and immediately call `rebalanceFunds()` to force the vault to trade at the manipulated rate.
2. **Double Fee Drain:** Each forced rebalance incurs Uniswap trading fees twice (0.3% on divest + 0.3% on reinvest). An attacker can loop this process to systematically drain vault funds.
3. **No User Interaction Required:** The attack does not depend on organic user traffic. The vault can be drained even during periods of low activity.

Proof of Concept:

```
1 function test_H04_PublicRebalanceEnablesMEV() public {
2     // 1. Setup: Guardian with 50% Hold / 50% Uniswap allocation
3     weth.mint(MINT_AMOUNT, guardian);
4     vm.startPrank(guardian);
5     weth.approve(address(vaultGuardians), MINT_AMOUNT);
6     address wethVaultAddr = vaultGuardians.becomeGuardian(
7         AllocationData(500, 500, 0));
8     VaultShares wethVault = VaultShares(wethVaultAddr);
9     vm.stopPrank();
10
11     address mevBot = makeAddr("mev_bot");
12
13     // Prove that ANYONE can call rebalanceFunds() without permission
14     vm.prank(mevBot);
15     wethVault.rebalanceFunds();
16
17     console2.log("POC H-04 Success: rebalanceFunds() is public and uses
18         0 slippage");
19     console2.log("- Deposit window: Vulnerable (amountOutMin = 0)");
20     console2.log("- Withdraw window: Vulnerable (amountOutMin = 0)");
21     console2.log("- Rebalance window: Vulnerable + Permissionless +
22         Double Fee");
23 }
```

Recommended Mitigation:

- Change `rebalanceFunds()` visibility to `external` and restrict it to the `onlyGuardian` modifier.
- Implement a cooldown mechanism (e.g., 1 hour between rebalances) to prevent rapid looping attacks.
- Integrate a TWAP oracle or slippage tolerance parameter to protect against sudden price manipulation during rebalances.

[H-05] Dust LP token donation bricks all withdrawals (DoS)**Description:**

The `divestThenInvest` modifier in `VaultShares.sol` checks if the vault holds any Uniswap LP tokens. If the balance is greater than zero, it attempts to divest (remove liquidity) before allowing the requested operation (e.g., `withdraw`, `redeem`) to proceed.

However, this check is vulnerable to “dust” attacks. An attacker can transfer a negligible amount (e.g., 1 wei) of the LP token to the vault address. This forces the vault to attempt a divestment of 1 wei during every subsequent withdrawal.

When `_uniswapDivest` is called with 1 wei:

1. `removeLiquidity` returns (0, 0) due to rounding.
2. The function then attempts to swap the counterparty token with `amountIn = 0`.
3. On the real Uniswap V2 Router, `swapExactTokensForTokens` requires `amountIn > 0` and will revert.

Impact:

A single transaction costing minimal gas (transferring 1 wei) can permanently brick the vault's withdrawal functionality. All user funds become locked because every attempt to `withdraw` or `redeem` will trigger the faulty divestment logic and revert.

Proof of Concept:

```
1 function test_H05_DustLPDonationDoS() public {
2     // 1. Setup: Guardian with 100% Hold (No Uniswap allocation)
3     weth.mint(MINT_AMOUNT, guardian);
4     vm.startPrank(guardian);
5     weth.approve(address(vaultGuardians), MINT_AMOUNT);
6     address wethVaultAddr = vaultGuardians.becomeGuardian(
7         AllocationData(1000, 0, 0));
8     VaultShares wethVault = VaultShares(wethVaultAddr);
9     vm.stopPrank();
10
11     // 2. User deposits funds
12     weth.mint(DEPOSIT_AMOUNT, user);
13     vm.startPrank(user);
14     weth.approve(address(wethVault), DEPOSIT_AMOUNT);
15     wethVault.deposit(DEPOSIT_AMOUNT, user);
16     vm.stopPrank();
17
18     // 3. Attacker sends 1 wei of LP token (Dust) to the vault
19     address lpToken = wethVault.getUniswapLiquidityToken();
20     ERC20Mock(lpToken).mint(1 wei, attacker);
21
22     vm.startPrank(attacker);
23     IERC20(lpToken).transfer(address(wethVault), 1 wei);
24     vm.stopPrank();
25
26     console2.log("LP Balance in Vault after dust attack:", IERC20(
27         lpToken).balanceOf(address(wethVault)));
28
29     // 4. Simulate Real Uniswap Revert
30     address router = address(uniswapRouter);
31     vm.mockCallRevert(
32         router,
33         abi.encodeWithSelector(IUniswapV2Router01(uniswapRouter).
34             swapExactTokensForTokens.selector),
35         "UniswapV2Router: INSUFFICIENT_INPUT_AMOUNT"
```

```
33     );
34
35     // 5. User attempts to withdraw -> Should Revert
36     vm.startPrank(user);
37     vm.expectRevert("UniswapV2Router: INSUFFICIENT_INPUT_AMOUNT");
38     wethVault.withdraw(DEPOSIT_AMOUNT / 2, user, user);
39     vm.stopPrank();
40
41     console2.log("POC H-05 Success: Withdrawal reverted due to Dust LP
42     DoS");
43 }
```

Recommended Mitigation:

- Implement a minimum threshold check before calling `_uniswapDivest` (e.g., `if (balance > MINIMUM_LIQUIDITY_THRESHOLD)`).
- Alternatively, allow the guardian or an admin to “sweep” dust LP tokens from the vault to clear the blockage.

[H-06] Hardcoded stake price of 10 ether makes USDC guardian registration impossible

Description:

The `s_guardianStakePrice` in `VaultGuardiansBase.sol` is hardcoded to 10 ether (10 * 10¹⁸ = 10¹⁹ units). This value is used directly when registering a guardian for any token, regardless of that token’s decimal precision.

```
1 // VaultGuardiansBase.sol line 70
2 uint256 internal s_guardianStakePrice = 10 ether; // 10^19 units
3
4 // VaultGuardiansBase.sol lines 301-306
5 function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
6     private returns (address) {
7     // ...
8     token.safeTransferFrom(msg.sender, address(this),
9         s_guardianStakePrice);
10    // ...
11    tokenVault.deposit(s_guardianStakePrice, msg.sender);
12 }
```

Impact:

The stake amount is interpreted in the token’s native decimal format, not normalized. This creates catastrophic discrepancies:

Token	Decimals	Stake Required (units)	Real-World Value
WETH	18	10^{19}	~10 WETH (~\$18,000 at \$1,800/WETH)
LINK	18	10^{19}	~10 LINK (~\$140 at \$14/LINK)
USDC	6	10^{19}	10,000,000,000,000 USDC (\$10 trillion)

Real-World Context:

- Total USDC circulating supply (May 2026): ~\$32 billion
- Required stake: \$10 trillion
- **Required stake is 312x the entire USDC supply**

No entity on Earth possesses 10 trillion USDC. This makes it **mathematically impossible** to register as a USDC vault guardian. The USDC vault functionality is completely broken from deployment.

Proof of Concept (Code Analysis):

1. User calls `becomeTokenGuardian` with `token = USDC` (6 decimals).
2. `_becomeTokenGuardian` executes `token.safeTransferFrom(msg.sender, address(this),  $10^{19}$ )`.
3. USDC has only 6 decimals, so 10^{19} units represents $10^{19} / 10^6 = 10^{13} = 10$ trillion USDC.
4. Transaction reverts because user cannot possibly have 10 trillion USDC.
5. USDC guardian registration is permanently impossible.

Recommended Mitigation:

Normalize the stake price based on the token's decimal precision:

```
1 function _getNormalizedStakePrice(IERC20 token) internal view returns (
    uint256) {
2     uint8 decimals = IERC20Metadata(address(token)).decimals();
3     return 10 * 10**decimals;
4 }
```

Alternatively, peg the stake price to a USD value and calculate the equivalent token amount using a price oracle.

[H-07] mint() bypasses custom deposit economics (fees and investment strategy)**Description:**

The `VaultShares` contract overrides `deposit()` to implement custom economics: minting fees to the guardian and DAO, and investing funds into the strategy. However, the `mint()` function is **not overridden**, meaning it uses the base ERC4626 implementation which bypasses all custom logic.

Custom `deposit()` implementation:

```
1 function deposit(uint256 assets, address receiver) public override {
2     _deposit(_msgSender(), receiver, assets, shares);
3     _mint(i_guardian, shares / i_guardianAndDaoCut);
4     _mint(i_vaultGuardians, shares / i_guardianAndDaoCut);
5     _investFunds(assets);
6     return shares;
7 }
```

Base `mint()` (NOT overridden):

```
1 // OpenZeppelin ERC4626 default:
2 function mint(uint256 shares, address receiver) public virtual {
3     _deposit(_msgSender(), receiver, previewMint(shares), shares);
4     // NO fee minting
5     // NO _investFunds()
6 }
```

Impact:

Users can bypass the protocol's economic model by calling `mint()` directly instead of `deposit()`:

1. **Fee bypass:** Guardian and DAO receive no fees from `mint()` calls
2. **Strategy bypass:** Assets are not invested in Aave/Uniswap - they sit idle in the vault
3. **Yield impact:** User assets do not earn yield, reducing overall vault performance
4. **Free-rider problem:** Users can deposit without paying fees while still benefiting from other users' investments

Proof of Concept:

```
1 function test_H07_MintBypassesDepositEconomics() public {
2     // Setup and verification...
3     // User deposits via deposit() - fees are minted, funds invested
4     // User deposits via mint() - NO fees, NO investment
5     // Verify guardian/DAO did NOT receive additional shares from mint
6     ()
7 }
```

Recommended Mitigation:

Override `mint()` to include the same custom economics as `deposit()`:

```
1 function mint(uint256 shares, address receiver) public override {
2     uint256 assets = previewMint(shares);
```



```

3     _deposit(_msgSender(), receiver, assets, shares);
4     _mint(i_guardian, shares / i_guardianAndDaoCut);
5     _mint(i_vaultGuardians, shares / i_guardianAndDaoCut);
6     _investFunds(assets);
7     return shares;
8 }

```

[H-08] No timelock in governance allows immediate execution of malicious proposals

Description:

The `VaultGuardianGovernor` contract inherits standard OpenZeppelin Governor modules but **does not inherit `GovernorTimelockControl`**. This means proposals execute immediately after the voting period ends, with zero delay.

```

1 // VaultGuardianGovernor.sol
2 contract VaultGuardianGovernor is
3     Governor,
4     GovernorCountingSimple,
5     GovernorVotes,
6     GovernorVotesQuorumFraction
7 {
8     // MISSING: GovernorTimelockControl
9     // MISSING: queue() function
10    // MISSING: timelock() view function
11 }

```

Impact:

Scenario	With Timelock	Without Timelock (Current)
Malicious proposal passes	Users have 2-7 days to exit before execution	Execution immediate - zero exit window
Guardian changes strategy	Users can withdraw if they disagree	Too late - strategy already changed
Emergency parameter change	Delayed but safe	Fast but risky

1. **Zero exit window:** Proposals execute immediately after voting period ends
2. **Users trapped:** No time to withdraw if a malicious proposal passes
3. **DAO controls critical params:** Guardian stake price, fee cuts, allocation limits

Proof of Concept:

```
1 function test_H08_NoTimelockInGovernance() public {
2     // Deploy Governor (NO timelock)
3     VaultGuardianToken govToken = new VaultGuardianToken();
4     VaultGuardianGovernor governor = new VaultGuardianGovernor(govToken
5         );
6     // Create and execute proposal
7     // Verify execution is IMMEDIATE - no queue(), no timelock delay!
8 }
```

Recommended Mitigation:

Add `GovernorTimelockControl` to the inheritance chain:

```
1 import {GovernorTimelockControl} from "@openzeppelin/contracts/
   governance/extensions/GovernorTimelockControl.sol";
2 import {TimelockController} from "@openzeppelin/contracts/governance/
   TimelockController.sol";
3
4 contract VaultGuardianGovernor is
5     Governor, GovernorCountingSimple, GovernorVotes,
6     GovernorVotesQuorumFraction,
7     GovernorTimelockControl
8 {
9     constructor(IVotes _voteToken, TimelockController _timelock)
10         Governor("VaultGuardianGovernor")
11         GovernorVotes(_voteToken)
12         GovernorVotesQuorumFraction(4)
13         GovernorTimelockControl(_timelock)
14     {}
15 }
```

Medium

[M-01] Immutable Aave Pool Address Prevents Adapter Migration When Aave Pool Changes

Description:

The `AaveAdapter` stores the Aave Pool address in an immutable variable, `i_aavePool`, at deployment time. Because immutable variables cannot be changed after deployment, the protocol has no way to update the Aave Pool address if Aave migrates, upgrades, or replaces its Pool contract in the future.

Impact:

If Aave migrates to a new Pool address, the adapter will continue interacting with the old Pool address. This can cause vault functionality that depends on Aave to become partially or fully unusable.

Recommended Mitigation:

Make the Pool address configurable through an owner or governance-controlled setter:

```
1  IAavePool public aavePool;
2
3  function setAavePool(address newPool) external onlyOwner {
4      if (newPool == address(0)) revert InvalidAddress();
5      address oldPool = address(aavePool);
6      aavePool = IAavePool(newPool);
7      emit AavePoolUpdated(oldPool, newPool);
8  }
```

[M-02] Missing Aave Rewards Claiming Mechanism Causes Protocol Yield Loss**Description:**

The [AaveAdapter](#) integrates with Aave by supplying assets into the Aave Pool, but it does not implement any mechanism to claim rewards accrued from Aave V3 incentive programs through the [RewardsController](#).

Impact:

Any Aave rewards accrued by the adapter may remain permanently unclaimed. Users do not receive the full yield generated by the Aave position.

Recommended Mitigation:

Add a controlled reward-claiming mechanism to the [AaveAdapter](#) that stores the [RewardsController](#) address and exposes a function to claim rewards.

[M-03] Governor Quorum Configuration Mismatch Can Weaken Governance Security**Description:**

[VaultGuardianGovernor](#) inherits from OpenZeppelin [GovernorVotesQuorumFraction](#) and configures the quorum fraction as 4. The inline comment states that the minimum voting power required for a valid vote is 20%, but [GovernorVotesQuorumFraction\(4\)](#) actually means only 4% quorum is required.

Impact:

Proposals can become valid with only 40,000 votes instead of the expected 200,000 votes (for 1,000,000 total supply). The quorum requirement is five times lower than documented.

Recommended Mitigation:

If the intended quorum is 20%, use `GovernorVotesQuorumFraction(20)`.

[M-04] Guardians Can Be Created Without Paying the Documented Entry Fee**Description:**

`VaultGuardiansBase` defines a `GUARDIAN_FEE` constant of `0.1 ether` and a custom error `VaultGuardiansBase__FeeTooSmall`. However, `becomeGuardian()` is not marked as `payable` and does not check `msg.value`.

Impact:

Users can bypass the documented guardian entry fee entirely. The existence of `GUARDIAN_FEE` strongly indicates the protocol intended to enforce an ETH fee, but the enforcement logic was never implemented.

Recommended Mitigation:

Make `becomeGuardian()` payable and enforce the required guardian fee.

[M-05] Repeated Guardian Registration Orphans Previously Tracked Vault**Description:**

`VaultGuardiansBase::_becomeTokenGuardian()` stores the guardian vault address in the `s_guardians` mapping. The function does not check whether `msg.sender` is already registered as a guardian for the same token. Repeated calls overwrite the previous mapping entry, leaving the old vault active but no longer reachable through the protocol's canonical guardian registry.

Impact:

After the overwrite, the old vault still exists and can remain active, but the protocol no longer tracks it. `quitGuardian()` only operates on the latest registered vault.

Recommended Mitigation:

Prevent duplicate guardian registration for the same guardian/token pair:

```
1 if (address(s_guardians[msg.sender][token]) != address(0)) {  
2     revert VaultGuardiansBase__AlreadyGuardian(msg.sender, address(  
3         token));  
3 }
```

[M-06] Performance Fee Is Charged Against Principal Instead of Profit**Description:**

`VaultShares.deposit()` mints additional shares to the guardian and the Vault Guardians protocol on every deposit. The README describes the guardian reward model as performance-based, but the implementation does not measure profit, yield, or strategy performance before minting fee shares.

Impact:

Users are diluted on every deposit even if the guardian has generated no yield. The fee is charged purely because a deposit occurred, not because profit was generated.

Recommended Mitigation:

If the intended design is a true performance fee, remove fee minting from `deposit()` and only mint fee shares when actual profit has been realized using a high-watermark or checkpoint mechanism.

Low**[L-01] Missing Token Uniqueness Validation Allows Duplicate Supported Token Configuration****Description:**

The `VaultGuardiansBase` constructor does not validate that `weth`, `tokenOne`, and `tokenTwo` are unique addresses. If two token roles are configured with the same address, the protocol can be deployed with an invalid configuration.

Recommended Mitigation:

Validate that all configured token addresses are non-zero and unique during construction.

[L-02] GUARDIAN_FEE Constant Is Defined But Never Used**Description:**

`VaultGuardiansBase` defines a `GUARDIAN_FEE` constant of `0.1 ether` and a fee-related custom error, but the constant is never enforced in `becomeGuardian()`.

Recommended Mitigation:

Either implement the fee or remove the unused constant and error.

[L-03] i_uniswapFactory Is Stored But Not Used In UniswapAdapter**Description:**

`UniswapAdapter` stores the Uniswap factory address in an immutable variable but never uses it after construction.

Recommended Mitigation:

Remove `i_uniswapFactory` if it is not needed.

[L-04] _aaveDivest() Named Return Value Is Never Assigned**Description:**

`AaveAdapter::_aaveDivest()` declares a named return value `amountOfAssetReturned` but never assigns it, always returning the default value of 0.

Recommended Mitigation:

Assign the return value from `i_aavePool.withdraw()`.

[L-05] Typographical Errors In Event Names Can Break Off-Chain Indexing**Description:**

`VaultGuardiansBase` defines `GaurdianRemoved` (should be `GuardianRemoved`) and `DinvestedFromGuardian` (should be `DivestedFromGuardian`).

Recommended Mitigation:

Rename the events before deployment.

[L-06] GuardianAdded Event Does Not Include The Created Vault Address**Description:**

`GuardianAdded` event only includes the guardian address and token address, but not the created `VaultShares` vault address.

Recommended Mitigation:

Include the vault address in the event:

```
1 event GuardianAdded(address indexed guardianAddress, IERC20 indexed token, address indexed vaultAddress);
```

[L-07] FundsInvested() Event Does Not Emit Useful Investment Data**Description:**

The `FundsInvested()` event is declared with no parameters and emits no useful data despite investment calculations being available in the same function.

Recommended Mitigation:

Include relevant investment data in the event.

[L-08] updateGuardianStakePrice Emits The Same Value For Old And New Stake Price**Description:**

`VaultGuardians.updateGuardianStakePrice` updates the stake price before emitting the event, causing both values to be identical.

Recommended Mitigation:

Cache the old value before updating state.

[L-09] updateGuardianAndDaoCut Emits The Wrong Event Type**Description:**

`VaultGuardians.updateGuardianAndDaoCut` updates the guardian and DAO cut value but emits a stake price update event instead of a fee-related event.

Recommended Mitigation:

Create and emit a dedicated event for guardian/DAO cut updates.

Informational**[I-01] Hard dependency on Aave/Uniswap in withdrawal and deposit paths****Description:**

The `divestThenInvest` modifier is applied to all withdrawal and deposit functions. If Aave is paused or Uniswap lacks liquidity, withdrawals revert even if the vault holds sufficient idle assets.

Assessment:

This is a known risk inherent to ERC4626 yield-bearing vaults that integrate with external protocols.

[I-02] _uniswapDivest() returns incorrect value, ignoring direct liquidity return**Description:**

The `_uniswapDivest` function ignores `tokenAmount` (direct return from `removeLiquidity`) and only returns `amounts[1]` (swapped amount).

[I-03] Single-Step Ownership Transfer Can Cause Accidental Ownership Loss**Description:**

The protocol uses OpenZeppelin `Ownable` instead of `Ownable2Step`, making ownership transfers immediate and irreversible if sent to the wrong address.

Recommended Mitigation:

Use `Ownable2Step` for two-step ownership transfer.

[I-04] Temporary Swap Path Is Stored In Contract Storage**Description:**

`UniswapAdapter` stores the swap path in a state variable `s_pathArray` despite it being temporary data that only needs to exist during a single function call.

Recommended Mitigation:

Use a local memory array instead of storage.